Assignment 1 — Build a RAG System for Quarterly Financial Reports

Level: Beginner · **Effort:** 8–10 hours · **Submission:** GitHub repository link + 3-minute demo video

1. What you are building (in plain language)

A large language model like GPT-4o has never seen your company's quarterly results PDF. If you simply ask it "what was the profit last quarter?", it will either refuse or make something up.

RAG (Retrieval Augmented Generation) fixes this in five steps:

1. **Read** the PDF and pull out its text.
2. **Chunk** — cut that text into small pieces of a few hundred words each, because you cannot send a 60-page PDF to the model on every question.
3. **Embed** — convert each chunk into a list of numbers (called a vector) that captures its meaning.
4. **Store** those vectors in a vector database (ChromaDB) so they can be searched by meaning, not by exact keyword.
5. **Ask** — when the user types a question, convert the question into a vector too, pull the 4–5 closest chunks from Chroma, and hand *only those chunks plus the question* to GPT-4o. The model answers using the text you gave it.

That is the entire system. Nothing more is required in this assignment.

2. The business scenario

You have joined the research desk of an investment advisory firm. Every quarter, analysts spend hours reading through results announcements to answer routine questions: what was revenue, how did margins move, what did management say about demand. Your job is to build an internal tool where an analyst uploads the quarterly PDFs, asks a question in plain English, and gets an answer with the source page cited so they can verify it.

3. Data you must collect yourself

Pick **one listed company** and download **3 to 4 consecutive quarterly results PDFs** for it.

- Good sources: the company's own website under *Investor Relations → Financial Results / Quarterly Results*, or the stock exchange filing page (BSE, NSE, or SEC EDGAR for US companies).
- Prefer the **press release** or **fact sheet** PDF over the full audited statement. Press releases contain management commentary, which makes for more interesting questions.
- Suggested companies (pick any one, or your own): Infosys, TCS, HDFC Bank, Reliance Industries, Tata Motors, Asian Paints, Apple, Microsoft.

> **Important check before you start coding:** open each PDF and try to select the text with your mouse. If the text cannot be selected, the file is a scanned image and normal PDF readers will return empty text. Choose a different file or a different company. Do not attempt OCR in this assignment.

Optional extra: you may use the `yfinance` Python library to pull the share price or market capitalisation and show it alongside the answer. This is a nice-to-have, not a requirement, and it must not replace the PDF-based answering.

4. Technical requirements (all mandatory)

Component	What you must use
Language	Python 3.10 or above
PDF reading	`pypdf`, `pdfplumber`, or LangChain's `PyPDFLoader`
Chunking	Recursive character splitting, chunk size 800–1200 characters, overlap 100–200 characters
Embeddings	OpenAI `text-embedding-3-small`
Vector database	**ChromaDB**, persisted to a folder on disk
Answering model	**GPT-4o**, temperature between 0 and 0.2
Orchestration	LangChain **or** LangGraph **or** the plain `openai` SDK — your choice, all three are acceptable
User interface	Streamlit or Gradio
API key handling	Stored in a `.env` file, loaded with `python-dotenv`, and `.env` listed in `.gitignore`

5. Features your application must have

1. **Upload** — the user can upload one or more PDF files from the interface.

2. **Index** — a button that processes the uploaded files and shows a confirmation such as "*3 files processed, 214 chunks stored*".
3. **Ask** — a text box for the question and a button to submit it.
4. **Answer** — the answer from GPT-4o displayed clearly on screen.
5. **Sources** — under every answer, show which chunks were used, with the file name and page number. An analyst must be able to open the PDF and verify the number.
6. **Honest refusal** — if the answer is not present in the uploaded documents, the app must say so instead of guessing. You achieve this through your system prompt, for example: "*Answer only from the context provided below. If the context does not contain the answer, reply that the information is not available in the uploaded documents.*"
7. **Persistence** — stop the app, start it again, and the previously indexed documents must still be searchable without re-uploading. This is what "persisted to disk" means.

6. Optional bonus — a FastAPI backend

For extra marks, move the logic out of the interface and into a FastAPI service with these endpoints:

Method	Endpoint	Input	Output
POST	`/ingest`	One or more PDF files	`{"files": 3, "chunks": 214}`
POST	`/ask`	`{"question": "...", "top_k": 4}`	`{"answer": "...", "sources": [{"file": "...", "page": 7}]}`
GET	`/stats`	—	Collection name, total chunks, embedding model, LLM model

The Streamlit or Gradio interface then calls these endpoints over HTTP instead of doing the work itself. Run the service with `uvicorn main:app --reload` and confirm every endpoint works from the automatic documentation page at `http://localhost:8000/docs`.

7. Test questions — your app must handle all of these

Adapt the wording to the company you chose. Record the answer your app produced for each one and include this in your README.

1. What was total revenue in the most recent quarter you loaded?
2. Compare net profit across all the quarters you loaded. Which was highest?
3. How did revenue in the latest quarter compare with the same quarter of the previous year?
4. What did management say about the demand outlook or business environment?
5. Which business segment or geography grew fastest, and by how much?
6. What was the operating margin in each quarter, and is the trend rising or falling?
7. Was any dividend declared? State the amount per share and the record date.
8. What risks, headwinds, or challenges are mentioned in the documents?
9. Give me a three-line summary of the latest quarter for a client email.
10. **Deliberate trap question:** ask something the documents definitely do not contain, for example "*What is the CEO's personal shareholding in 2015?*" Your app must reply that the information is not available, not invent a figure. This question carries marks.

8. What you must submit

A public GitHub repository with the following structure:

```
finance-rag/
├── app.py                  # Streamlit / Gradio interface
├── ingest.py               # load, chunk, embed, store in Chroma
├── rag.py                  # retrieve + prompt + call GPT-4o
├── api/main.py             # optional FastAPI backend
├── data/                   # the quarterly PDFs you downloaded
├── chroma_db/              # persisted vector store (add to .gitignore if large)
├── requirements.txt
├── .env.example            # OPENAI_API_KEY=your_key_here
├── .gitignore              # must include .env
└── README.md
```

Your `README.md` must contain: the company you chose and links to the PDFs, setup and run instructions that a stranger can follow, your chunk size and overlap with a one-line reason for the choice, screenshots of the working app, the ten questions above with the answers your app gave, and an honest note on what did not work well.

Also record a **3-minute video** showing upload, indexing, three questions being answered, and the trap question being refused correctly.

9. How you will be marked

Area	Marks
Ingestion pipeline — loading, chunking, embedding without errors	20
ChromaDB used correctly and survives a restart	20
Answer quality, with sources and page numbers shown	20
Working user interface	15
Correct refusal on the trap question	10
Code quality, README, and repository hygiene	15
Bonus: FastAPI backend with all three endpoints working	+15

10. Rules, hints, and common mistakes

- **Keep it simple.** No re-ranking, no hybrid search, no agents, no fine-tuning. Anything beyond the requirements above earns zero extra marks and usually breaks something.
- **Never put your API key in code.** A key committed to GitHub is detected and disabled within minutes, and you will have to generate a new one.
- **If answers are wrong, check retrieval before blaming the model.** Print the chunks that were retrieved for the question. Nine times out of ten the model answered correctly from the wrong chunks, which means your chunking or your `top_k` is the problem, not GPT-4o.
- **Financial tables come out messy.** PDF tables lose their alignment when converted to plain text. This is normal and expected at this level. Increasing chunk size to around 1200 characters often keeps a whole table inside one chunk and improves answers.
- **Watch your spending.** `text-embedding-3-small` is very cheap, but re-indexing the same files in a loop while debugging adds up. Index once, then test questions.
- **Numbers must be checked by hand.** Before you submit, open the PDF and verify at least three figures your app reported. Marks are for correct answers, not confident-sounding ones.

11. Submission checklist

☐ 3–4 quarterly PDFs downloaded and stored in `data/` ☐ Chunking works and reports chunk count ☐ Chroma folder persists after restart ☐ GPT-4o answers with sources and page numbers ☐ Trap question refused correctly ☐ `.env` is in `.gitignore` ☐ README complete with screenshots and the ten answers ☐ 3-minute demo video recorded ☐ Repository link submitted